REGULAR EXPRESSIONS(re module)

→it specifies a set of strings(patterns) that matches it.

→to understand RE analogy, metacharacters are useful.

there are 14 metacharacters:

metacharacter	description
\	Used to drop the special meaning of character
[]	Represent a character class
^	Matches the end
$	Matches the end
.	Matches any character except newline
?	Matches zero (or) one occurrence
|(or)	Matches with any of the characters separated by it
*	Any no.of occurrence(including zero occurrences)
+	One or more occurences
{}	Indicate no.of occurrences of a preceding RE to match
()	Enclose a group of RE's

functions:

1.compile():

this functions is compiled into pattern objects.

Example:

```
>>> import re
>>> p=re.compile('[a-g]')
>>> print(p.findall('i love my country'))
['e', 'c']
>>> print(p.findall('today we are discussing regular expressions'))
['d', 'a', 'e', 'a', 'e', 'd', 'c', 'g', 'e', 'g', 'a', 'e', 'e']
>>>
```

→MetaCharacters backslash has a very important role as it signals various sequences.

metacharacters	Description
\d	Matches any decimal digit , this is equivalent to the set class[0-9]
\D	Matches any non-digit character
\s	Matches any white space character
\S	Matches any non-white space character
\w	Matches any alphanumeric character, this is equivalent to the class[a-z,A-Z,0-9]
\W	Matches any non-alphanumeric character

Example:

```
>>> import re
>>> p=re.compile('\d')
>>> print(p.findall("i went to hyd at 11'o clock on 2nd july 2018"))
```

```
['1', '1', '2', '2', '0', '1', '8']
>>> p=re.compile('\d+')
>>> print(p.findall("i went to hyd at 11'o clock on 2nd july 2018"))
['11', '2', '2018']
>>> p=re.compile('\D')
>>> print(p.findall("i went to hyd at 11'o clock on 2nd july 2018"))
['i', ' ', 'w', 'e', 'n', 't', ' ', 't', 'o', ' ', 'h', 'y', 'd', ' ', 'a', 't', ' ', "'", 'o', ' ', 'c', 'l', 'o', 'c', 'k', ' ', 'o', 'n', ' ', 'n', 'd',
' ', 'j', 'u', 'l', 'y', ' ']
>>> p=re.compile('\w')
>>> print(p.findall('i love my country'))
['i', 'l', 'o', 'v', 'e', 'm', 'y', 'c', 'o', 'u', 'n', 't', 'r', 'y']
>>> p=re.compile('\w+')
>>> print(p.findall('i love my country'))
['i', 'love', 'my', 'country']
>>> p=re.compile('\W')
>>> print(p.findall('i love my country'))
[' ', ' ', ' ']
>>>
>>> import re
>>> p=re.compile('\s')
>>> print(p.findall("i love my country"))
[' ', ' ', ' ']
>>> p=re.compile('\S')
>>> print(p.findall("i love my country"))
['i', 'l', 'o', 'v', 'e', 'm', 'y', 'c', 'o', 'u', 'n', 't', 'r', 'y']
>>>
>>> import re
>>> p=re.compile('ab*')
>>> print(p.findall('abababababababababababbbbbbba'))
['ab', 'ab', 'ab', 'ab', 'ab', 'ab', 'ab', 'ab', 'ab', 'ab', 'ab', 'abbbbbbb', 'a']
>>>
```

2.split():

it split by the occurrences of a character (or) a pattern upon finding that pattern, the remaining characters from the string are returned as part of the resulting list.

Syntax:

re.split(pattern,string,maxsplit=0,flags=0)

Note:

pattern-it denotes regular expression

String-pattern will be searched for and in which splitting occurs

Maxsplit-(a) if any non-zero value is provided then at most that many splits occurs.

(b)maxsplit=1, the the string will split once only

flag-it is optional

Example:

```
>>> from re import split
>>> print(split('\w','sushma,anil,mani'))
['', '', '', '', '', '', ',', '', '', '', ',', '', '', '', '']
>>> print(split('\w+','sushma anil mani chanti'))
['', ' ', ' ', ' ', '']
>>> print(split('\d+','on 8th sep 2018, at 2:30PM'))
['on ', 'th sep ', ', at ', ':', 'PM']
>>> import re
>>> print(re.split('[a-f]+','we are learning python concepts'))
['w', ' ', 'r', ' l', 'rning python ', 'on', 'pts']
>>> print(re.split('[a-f]+','we are learning PYTHON CONCEPTS',flags=re.IGNORECASE))
['w', ' ', 'r', ' l', 'rning PYTHON ', 'ON', 'PTS']
>>>
```

3.sub()(Substring):

Syntax:

re.sub(pattern,repl,string,count=0,flags=0)

this pattern is searched in the given string(3rd parameter), and open finding the substrig pattern is replaced by repl(2nd parameter), count and checks and maintains the no.of times this occurs.

Example:

```
>>> import re
>>> print(re.sub('su','**','sushma is learning python subject'))
**shma is learning python **bject
>>> print(re.sub('su','**','sushma is learning python SUBJECT',flags=re.IGNORECASE))
**shma is learning python **BJECT
```

4.subn():

Syntax:

re.subn(pattern,repl,string,count=0,flags=0)

it is similar to sub(), but it returns a tuple with count of total of replacement and the new string rather than just the string.

Example:

```
>>> import re
>>> print(re.subn('su','**','sushma is learning python subject'))
('**shma is learning python **bject', 2)
>>> print(re.subn('su','**','sushma is learning python SUBJECT',flags=re.IGNORECASE))
('**shma is learning python **BJECT', 2)
>>>
```

5.escape():

Syntax:

re.escape(string)

it returns string with all non-alphanumeric backslashed.

Example:

```
>>> import re
>>> print(re.escape("this is awesome even 1AM"))
this\ is\ awesome\ even\ 1AM
```